

Phase 1 API List

ODC — Crito Integration

Document Purpose: This document lists the Phase 1 APIs required for integration between Crito Smart PMS and ODC. Phase 1 work involves exposing these capabilities through a dedicated API service with standardized endpoints. Only slight amendments may be required to align request/response formats.

1. Architecture Overview

Question	Answer
Separate service or part of existing application?	A separate API service/project will be created. It will not be embedded into the existing staff web application.
Does this require new PMS development?	No. All core functionality already exists in the PMS. The API project will expose existing PMS capabilities through a standardized integration layer.
How is it configured per property?	Each property is identified by a Property / Location ID in PMS. Per-property settings (payment gateway, company details, terms, etc.) are already maintained in PMS and will be respected by the API.
Suggested base URL	<code>https://{host}/integration/api/v1</code>

2. Phase 1 API List

1. Authentication / Token

API Name	Get Access Token
Purpose	Authenticate integration client and return access token
Endpoint / Method	POST /auth/token
Input Fields	Client ID, Client Secret, Grant Type
Output Fields	Access Token, Token Type, Expiry

PMS Source	Existing integration and authentication setup in PMS
Status	Modification — to be exposed through new API structure

2. Property Details

API Name	Get Property / List Properties
Purpose	Return property information for sync
Endpoint / Method	GET /properties, GET /properties/{propertyId}
Input Fields	Property ID
Output Fields	Property ID, Name, Address, Status, Settings
PMS Source	Property / Location management
Status	Existing — available in PMS, to be exposed via API

3. Room / Unit Sync

API Name	List Units
Purpose	Sync rooms/units and their status
Endpoint / Method	GET /properties/{propertyId}/units
Input Fields	Property ID, Optional Filters
Output Fields	Unit ID, Room Number, Building, Floor, Unit Group, Status
PMS Source	Room and unit management
Status	Existing — available in PMS, to be exposed via API

4. Room Type / Unit Group Sync

API Name	List Unit Groups
Purpose	Sync room categories/types
Endpoint / Method	GET /properties/{propertyId}/unit-groups
Input Fields	Property ID
Output Fields	Unit Group ID, Name, Capacity, Features, Pricing Links
PMS Source	Room type management

Status	Existing — partially available today, to be standardized in API
--------	--

5. Availability by Date Range

API Name	Get Availability
Purpose	Return available inventory for a date range
Endpoint / Method	GET /properties/{propertyId}/availability
Input Fields	Property ID, From Date, To Date, Optional Unit Group
Output Fields	Unit Group, Available Count, Restrictions, Price
PMS Source	Availability and booking search
Status	Existing — slight amendment for date-range format

6. Reservation — Create

API Name	Create Reservation
Purpose	Create a new booking/reservation
Endpoint / Method	POST /properties/{propertyId}/reservations
Input Fields	Unit Group, Check-in, Check-out, Guest Details, Reference
Output Fields	Reservation ID, Confirmation Number, Status, Guest ID, Dates
PMS Source	Booking / reservation management
Status	Existing — booking creation already in PMS, to be exposed via API

7. Reservation — Get

API Name	Get Reservation
Purpose	Retrieve single reservation details
Endpoint / Method	GET /properties/{propertyId}/reservations/{reservationId}
Input Fields	Property ID, Reservation ID
Output Fields	Reservation Details, Guest, Dates, Status, Unit Assignment, Balance
PMS Source	Booking / reservation management

Status	Existing — available in PMS, to be exposed via API
--------	--

8. Reservation — List

API Name	List Reservations
Purpose	List reservations by property and filters
Endpoint / Method	GET /properties/{propertyId}/reservations
Input Fields	Property ID, Date Range, Status, Pagination
Output Fields	List of Reservation Summaries
PMS Source	Booking / reservation management
Status	Existing — available in PMS, to be exposed via API

9. Guest Details — Get

API Name	Get Guest
Purpose	Retrieve guest/resident profile
Endpoint / Method	GET /properties/{propertyId}/guests/{guestId}
Input Fields	Property ID, Guest ID
Output Fields	Name, Email, Phone, Nationality, Linked Reservations
PMS Source	Guest / person management
Status	Existing — available in PMS, to be exposed via API

10. Guest Details — Update

API Name	Update Guest
Purpose	Update guest profile details
Endpoint / Method	PUT /properties/{propertyId}/guests/{guestId}
Input Fields	Updatable Guest Fields
Output Fields	Updated Guest Profile
PMS Source	Guest / person management
Status	Existing — available in PMS, to be exposed via API

11. Room Assignment

API Name	Assign Unit to Reservation
Purpose	Assign physical room/unit to a reservation
Endpoint / Method	PUT /properties/{propertyId}/reservations/{reservationId}/assign-unit
Input Fields	Unit ID, Optional Notes
Output Fields	Assignment ID, Unit Details, Status
PMS Source	Room placement / assignment
Status	Existing — available in PMS, to be exposed via API

12. Check-in

API Name	Check In
Purpose	Mark reservation as checked in
Endpoint / Method	PUT /properties/{propertyId}/reservations/{reservationId}/check-in
Input Fields	Check-in Time, Optional Card Details
Output Fields	Status, Check-in Time
PMS Source	Check-in process
Status	Existing — available in PMS, to be exposed via API

13. Check-out

API Name	Check Out
Purpose	Mark reservation as checked out
Endpoint / Method	PUT /properties/{propertyId}/reservations/{reservationId}/check-out
Input Fields	Check-out Time, Optional Notes
Output Fields	Status, Check-out Time, Final Balance
PMS Source	Check-out process
Status	Existing — available in PMS, to be exposed via API

14. Cancellation

API Name	Cancel Reservation
Purpose	Cancel a reservation
Endpoint / Method	PUT /properties/{propertyId}/reservations/{reservationId}/cancel
Input Fields	Cancellation Reason
Output Fields	Status, Cancelled Date, Reason
PMS Source	Booking cancellation
Status	Existing — available in PMS, to be exposed via API

15. Balance

API Name	Get Folio / Balance
Purpose	Return charges, payments, and outstanding balance
Endpoint / Method	GET /properties/{propertyId}/reservations/{reservationId}/folio
Input Fields	Property ID, Reservation ID
Output Fields	Charges, Payments, Pending Balance, Total Balance, Currency
PMS Source	Invoicing and ledger
Status	Existing — slight amendment to present as folio view

16. Payment Link Generation

API Name	Create Payment Link
Purpose	Generate payment link for outstanding balance
Endpoint / Method	POST /properties/{propertyId}/reservations/{reservationId}/payment-link
Input Fields	Amount (Optional), Invoice Reference, Return URL
Output Fields	Payment Link URL, Amount, Expiry
PMS Source	Payment gateway / invoicing
Status	Existing — payment link already in PMS, to be exposed via API

17. Payment Status

API Name	Get Payment Status
Purpose	Check payment link or transaction status
Endpoint / Method	GET /properties/{propertyId}/payments/{paymentId}
Input Fields	Property ID, Payment ID
Output Fields	Status, Amount, Paid Date, Reference
PMS Source	Payment processing
Status	Existing — available in PMS, to be exposed via API

18. Outbound Webhooks — Reservation Events

API Name	Reservation Webhook Events
Purpose	Send reservation updates to Crito (create, update, assign, check-in, check-out, cancel)
Endpoint / Method	Outbound POST to Crito-configured webhook URL
Input Fields	Event Type, Reservation Data, Timestamp
Output Fields	Delivery Acknowledgment
PMS Source	Reservation lifecycle events
Status	Modification — event triggers exist in PMS, webhook dispatch to be wired

19. Outbound Webhooks — Payment Events

API Name	Payment Webhook Events
Purpose	Send payment updates to Crito (link created, completed, failed)
Endpoint / Method	Outbound POST to Crito-configured webhook URL
Input Fields	Event Type, Payment Data, Reservation Reference
Output Fields	Delivery Acknowledgment
PMS Source	Payment lifecycle events
Status	Modification — payment flow exists in PMS, webhook dispatch to be wired

3. Summary

Item	Detail
Total APIs (Phase 1)	19
Existing (to be exposed via API)	15
Modification (slight amendments)	4 (Authentication, Availability, Balance, Webhooks)
New PMS functionality required	None
Deployment approach	Separate API service using existing PMS backend
Per-property configuration	Via Property / Location ID and existing per-property settings in PMS